



Noakhali Science And Technology University

Lecture 3

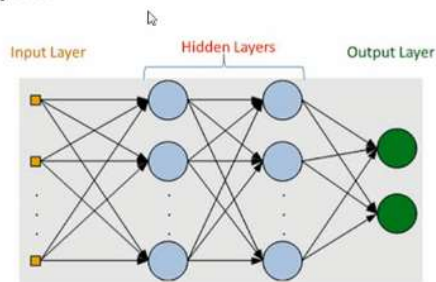
Backpropagation

PROF. DR. MD. ASHIKUR RAHMAN KHAN

Department of Information and Communication Engineering

Introduction to Multilayer Perceptron

Remember that from the definition of multilayer perceptron, there must be one or more hidden layers.



This means that in general, the layers of an MLP should be a minimum of three layers, since we have also the input and the output layer



Multilayer Perceptron MLP

- A multi-layer perceptron (MLP) has the same structure of a single layer perceptron with one or more hidden layers
- Let's assume we have one input layer, one hidden layer, and one output layer
- We use the inputs and weights to work out the activation for any node as we learned before (*i.e. weighted sum and transfer function*)
- This is easily achieved for the hidden layer as it has direct links to the actual input layer
- Therefore, we use the output from the hidden layer nodes to work out the activation for an output node (they are the inputs to the output layer nodes), *observe that the output layer is not directly connected to the input layer, so it knows nothing about it*
- Think of it as forward passing something from one layer to the next (propagate inputs by adding all the weighted inputs and then computing outputs using sigmoid threshold)

- Multilayer perceptrons have been applied to solve some difficult problems.
- This consist of input layer, one or more hidden layer and an output layer.
- The training of the network is done by the highly popular algorithm known as the error back-propagation algorithm.
- This algorithm is based on the error-correcting learning rule.
- Basically there are two passes through the different layers of the network: forward pass and the **backward pass**.

Backpropagation

The **demonstration of the limitations of single-layer** neural networks was a significant factor in the decline of interest in neural networks.

Multilayer perceptron (MLPs) are layered feed forward networks typically trained with static back propagation.

A Multilayer backpropagation network (feedforward net trained by backpropagation) **can be used to solve problems in many areas.**

A typical multi-layer network consists of an input, hidden and output layer, each fully connected to the next, with activation function

Although a single-layer net is severely limited in the mappings it can learn, a multilayer net can learn any continuous mapping to an arbitrary accuracy.

Dr. M.A.R. Khan, ICE, NSTU

Backpropagation

The continuous adjustment of the weights is a supervised learning process called 'backpropagation'.

Back propagation algorithm is stand on gradient descant which means that **it go downward on the error declination and adjust the weights for the minimum error.**

The training of a network by backpropagation involves two parts:

- ❑ **Forward pass**
- ❑ **Backward pass**

Forward pass: calculate the output corresponding to the inputs

Backward pass: propagates the error backwards through the network starting at the output units

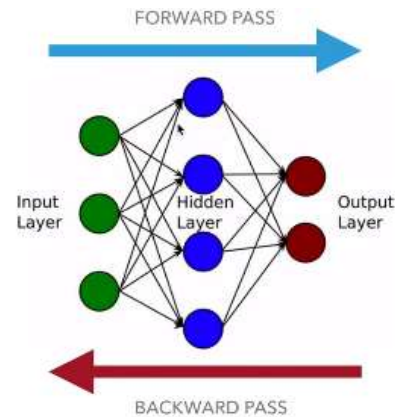
Dr. M.A.R. Khan, ICE, NSTU

Backpropagation

The weights are continuously updated every time the input patterns are presented to the network and the process continues until the error is at the lowest value.

If all the input patterns are propagated once through the network, it is called as cycle or epoch.

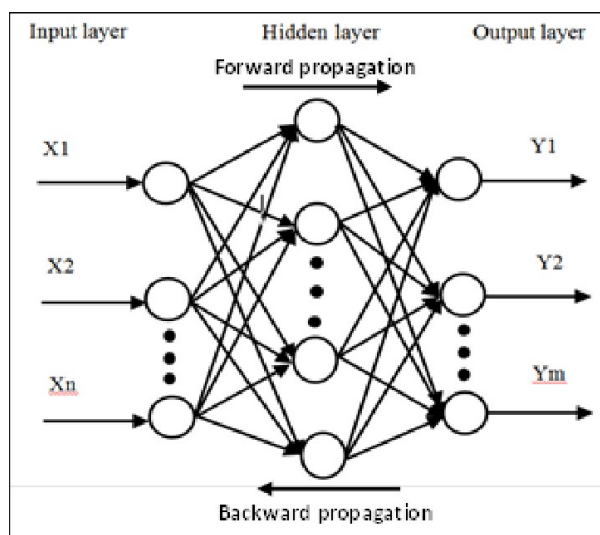
Typically, many epochs are required for training a backpropagation neural network.



Dr. M.A.R. Khan, ICE, NSTU

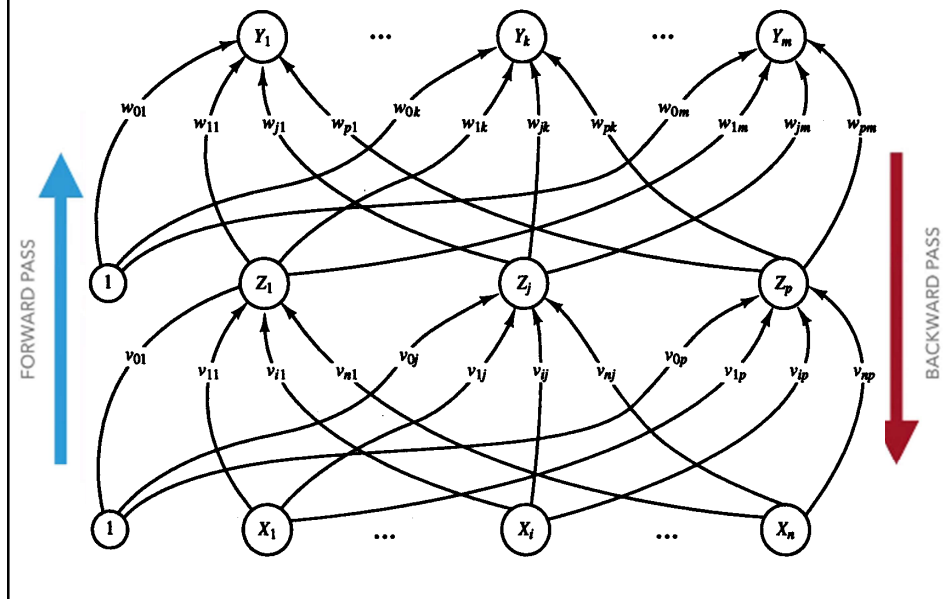
In Multilayer perceptron, there must be one or more hidden layer

More than one hidden layer may be beneficial for some applications, but one hidden layer is sufficient.



Backpropagation neural network with one hidden layer

Architecture



Algorithm of Backpropagation

Nomenclature

The nomenclature we use in the training algorithm for the backpropagation net is as follows:

x Input training vector:

$$\mathbf{x} = (x_1, \dots, x_i, \dots, x_n).$$

t Output target vector:

$$\mathbf{t} = (t_1, \dots, t_k, \dots, t_m).$$

δ_k Portion of error correction weight adjustment for w_{jk} that is due to an error at output unit Y_k ; also, the information about the error at unit Y_k that is propagated back to the hidden units that feed into unit Y_k .

δ_j Portion of error correction weight adjustment for v_{ij} that is due to the backpropagation of error information from the output layer to the hidden unit Z_j .

α Learning rate.

Algorithm of Backpropagation

Nomenclature (Contd.)

X_i	Input unit i : For an input unit, the input signal and output signal are the same, namely, x_i .
v_{0j}	Bias on hidden unit j .
Z_j	Hidden unit j : The net input to Z_j is denoted z_in_j : $z_in_j = v_{0j} + \sum_i x_i v_{ij}.$ The output signal (activation) of Z_j is denoted z_j : $z_j = f(z_in_j).$
w_{0k}	Bias on output unit k .
Y_k	Output unit k : The net input to Y_k is denoted y_in_k : $y_in_k = w_{0k} + \sum_j z_j w_{jk}.$ The output signal (activation) of Y_k is denoted y_k : $y_k = f(y_in_k).$

Dr. M.A.R. Khan, ICE, NSTU

Algorithm of Backpropagation

Activation Function

An activation function for a backpropagation net should have several important characteristics: it should be

- continuous
- differentiable, and
- monotonically non decreasing

One of the most typical activation functions is the binary sigmoid function, which has range of (0, 1) and is defined as

$$f_1(x) = \frac{1}{1 + \exp(-x)},$$

with

$$f'_1(x) = f_1(x)[1 - f_1(x)].$$

Dr. M.A.R. Khan, ICE, NSTU

Algorithm of Backpropagation

Another common activation function is bipolar sigmoid, which has range of (-1, 1) and is defined as

$$f_2(x) = \frac{2}{1 + \exp(-x)} - 1,$$

with

$$f'_2(x) = \frac{1}{2} [1 + f_2(x)][1 - f_2(x)].$$

Dr. M.A.R. Khan, ICE, NSTU

Derivation of Learning Rule

From Delta rule, w_{jk} denotes the weight between the hidden unit Z_j and the output unit Y_k .

The subscripts IJ are used for the weight between input unit Z_I and hidden unit, Z_j .

The squared error for a particular training pattern is

$$E = \sum_{j=1}^m (t_j - y_{in_j})^2. \quad y_{in_K} = \sum_j z_j w_{jK}$$

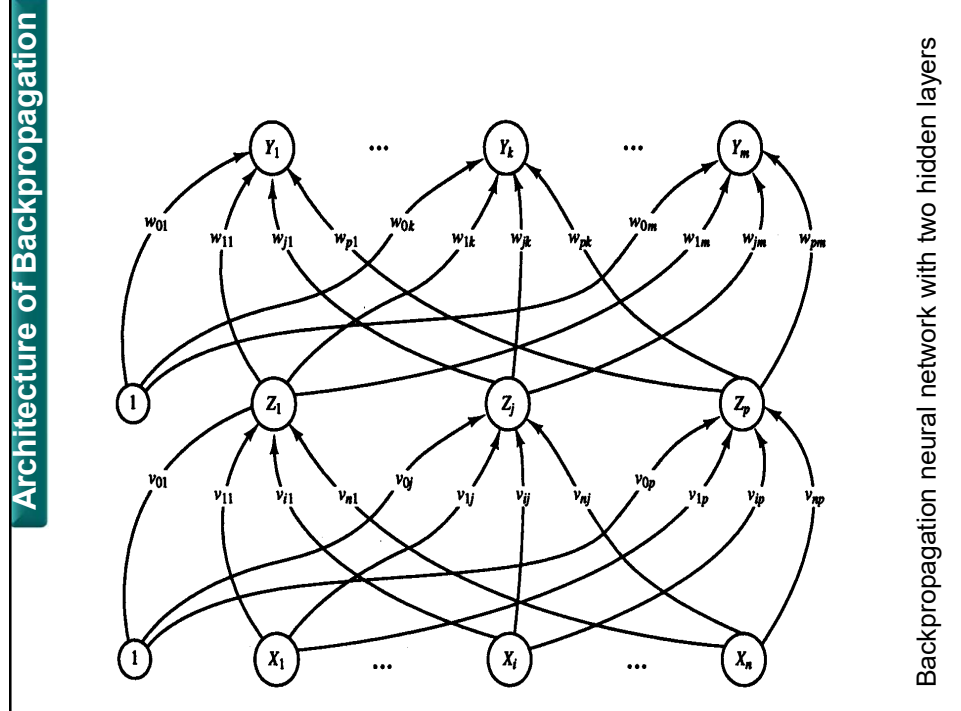
to find $f(y_{in_K})$.

The error (a function of the weights) to be minimized is

$$E = .5 \sum_k [t_k - y_k]^2.$$

By use of the chain rule, we have

$$\frac{\partial E}{\partial w_{JK}} = \frac{\partial}{\partial w_{JK}} .5 \sum_k [t_k - y_k]^2$$



Derivation of Learning Rule

$$\begin{aligned}
 \frac{\partial E}{\partial w_{JK}} &= \frac{\partial}{\partial w_{JK}} .5 \sum_k [t_k - y_k]^2 \\
 &= \frac{\partial}{\partial w_{JK}} .5 [t_K - f(y_{in_K})]^2 \\
 &= -[t_K - y_K] \frac{\partial}{\partial w_{JK}} f(y_{in_K}) \quad y_{in_K} = \sum_j z_j w_{jK} \\
 &= -[t_K - y_K] f'(y_{in_K}) \frac{\partial}{\partial w_{JK}} (y_{in_K}) \\
 &= -[t_K - y_K] f'(y_{in_K}) z_J.
 \end{aligned}$$

It is convenient to define δ_K :

$$\delta_K = [t_K - y_K] f'(y_{in_K}).$$

Derivation of Learning Rule

$$E = .5 \sum_k [t_k - y_k]^2.$$

For weights on connections to the hidden unit Z_J :

$$\begin{aligned} \frac{\partial E}{\partial v_{IJ}} &= - \sum_k [t_k - y_k] \frac{\partial}{\partial v_{IJ}} y_k \\ &= - \sum_k [t_k - y_k] f'(y_{in_k}) \frac{\partial}{\partial v_{IJ}} y_{in_k} \\ &= - \sum_k \delta_k \frac{\partial}{\partial v_{IJ}} y_{in_k} \quad y_{in_K} = \sum_j z_j w_{jK} \\ &= - \sum_k \delta_k w_{Jk} \frac{\partial}{\partial v_{IJ}} z_J \\ &= - \sum_k \delta_k w_{Jk} f'(z_{in_J}) [x_I]. \end{aligned}$$

Define:

$$\delta_J = \sum_k \delta_k w_{Jk} f'(z_{in_J})$$

Dr. M.A.R. Khan, ICE, NSTU

Derivation of Learning Rule

The updates for the weights to the output units (returning to the more common lower case subscripts) are given by:

$$\begin{aligned} \Delta w_{jk} &= - \alpha \frac{\partial E}{\partial w_{jk}} \\ &= \alpha [t_k - y_k] f'(y_{in_k}) z_j \\ &= \alpha \delta_k z_j; \end{aligned}$$

and for the weights to the hidden units:

$$\begin{aligned} \Delta v_{ij} &= - \alpha \frac{\partial E}{\partial v_{ij}} \\ &= \alpha f'(z_{in_j}) x_i \sum_k \delta_k w_{jk}, \\ &= \alpha \delta_j x_i. \end{aligned}$$

Dr. M.A.R. Khan, ICE, NSTU

Training Algorithm of Backpropagation

The Algorithm is as follows:

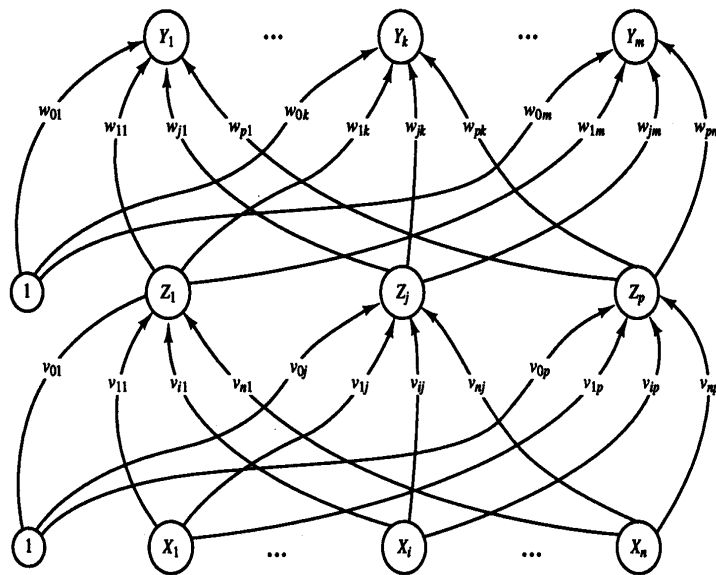
- Step 0.** Initialize weights.
(Set to small random values).
- Step 1.** While stopping condition is false, do Steps 2–9.
- Step 2.** For each training pair, do Steps 3–8.

Feedforward:

- Step 3.** Each input unit ($X_i, i = 1, \dots, n$) receives input signal x_i and broadcasts this signal to all units in the layer above (the hidden units).
- Step 4.** Each hidden unit ($Z_j, j = 1, \dots, p$) sums its weighted input signals,

$$z_{in_j} = v_{0j} + \sum_{i=1}^n x_i v_{ij},$$

applies its activation function to compute its output signal,



Training Algorithm of Backpropagation

$$z_j = f(z_in_j),$$

and sends this signal to all units in the layer above (output units).

Step 5. Each output unit ($Y_k, k = 1, \dots, m$) sums its weighted input signals,

$$y_in_k = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and applies its activation function to compute its output signal,

$$y_k = f(y_in_k).$$

Training Algorithm of Backpropagation

Backpropagation of error

Step 6. Each output unit ($Y_k, k = 1, \dots, m$) receives a target pattern corresponding to the input training pattern, computes its error information term,

$$\delta_k = (t_k - y_k)f'(y_in_k),$$

calculates its weight correction term (used to update w_{jk} later),

$$\Delta w_{jk} = \alpha \delta_k z_j,$$

calculates its bias correction term (used to update w_{0k} later),

$$\Delta w_{0k} = \alpha \delta_k,$$

and sends δ_k to units in the layer below.

Training Algorithm of Backpropagation

Step 7. Each hidden unit ($Z_j, j = 1, \dots, p$) sums its delta inputs (from units in the layer above),

$$\delta_{in_j} = \sum_{k=1}^m \delta_k w_{jk},$$

multiplies by the derivative of its activation function to calculate its error information term,

$$\delta_j = \delta_{in_j} f'(z_{in_j}), \quad \delta_J = \sum_k \delta_k w_{Jk} f'(z_{in_J})$$

calculates its weight correction term (used to update v_{ij} later),

$$\Delta v_{ij} = \alpha \delta_j x_i,$$

and calculates its bias correction term (used to update v_{0j} later),

$$\Delta v_{0j} = \alpha \delta_j.$$

Training Algorithm of Backpropagation

Update weights and biases:

Step 8. Each output unit ($Y_k, k = 1, \dots, m$) updates its bias and weights ($j = 0, \dots, p$):

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}.$$

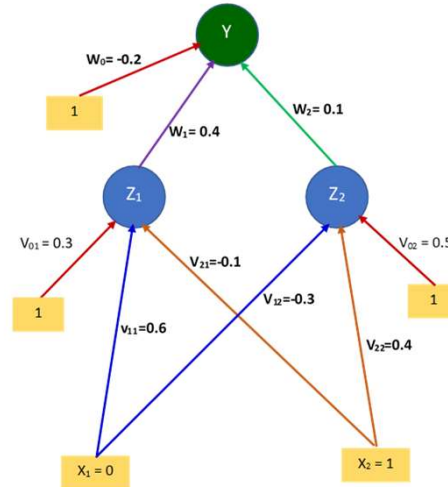
Each hidden unit ($Z_j, j = 1, \dots, p$) updates its bias and weights ($i = 0, \dots, n$):

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \Delta v_{ij}.$$

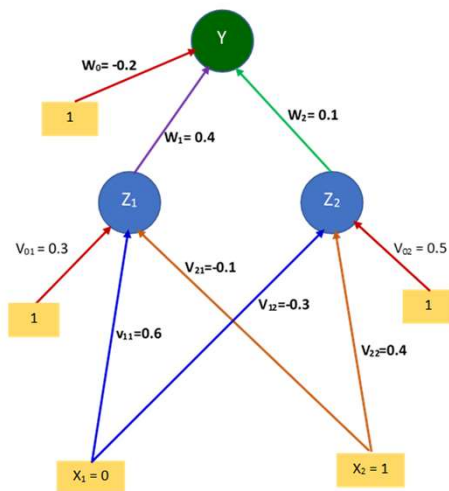
Step 9. Test stopping condition.

Example # 1

Use the backpropagation algorithm to solve the problem given in the following figure, where $[x_1 \ x_2] = [0 \ 1]$, $[t] = [1]$, learning rate = 0.25. Use sigmoid function as activation function.



Example # 1 (Contd.)



Example # 1 (Contd.)

Calculate net input to z_1 (hidden) layer

$$\begin{aligned} z_{in1} &= v_{01} + x_1 v_{11} + x_2 v_{21} \\ &= 0.3 + 0 \times 0.6 + 1 \times -0.1 \\ &= 0.2 \end{aligned}$$

The net input to Z_j is denoted z_{in_j} :

$$z_{in_j} = v_{0j} + \sum_i x_i v_{ij}.$$

For z_2 layer

$$\begin{aligned} z_{in2} &= v_{02} + x_1 v_{12} + x_2 v_{22} \\ &= 0.5 + 0 \times -0.3 + 1 \times 0.4 \\ &= 0.9 \end{aligned}$$

The output signal (activation) of Z_j is denoted z_j :

$$z_j = f(z_{in_j}).$$

Applying activation function to calculate output,

$$\begin{aligned} z_1 &= f(z_{in1}) = \frac{1}{1+e^{-z_{in1}}} = \frac{1}{1+e^{-0.2}} = 0.5498 \\ z_2 &= f(z_{in2}) = \frac{1}{1+e^{-z_{in2}}} = \frac{1}{1+e^{-0.9}} = 0.7109 \end{aligned}$$

Example # 1 (Contd.)

Calculate net input to output layer

For y layer

$$y_{in_k} = w_{0k} + \sum_j z_j w_{jk}.$$

$$\begin{aligned} y_{in} &= w_0 + z_1 w_1 + z_2 w_2 \\ &= -0.2 + 0.5498 \times 0.4 + 0.7109 \times 0.1 \\ &= 0.09101 \end{aligned}$$

Calculate the output,

$$y_k = f(y_{in_k}).$$

Applying activation function to calculate output,

$$y = f(y_{in}) = \frac{1}{1+e^{-y_{in}}} = \frac{1}{1+e^{-0.09101}} = \mathbf{0.5221}$$

Compute error portion δ_k ,

$$\begin{aligned}\delta_k &= (t_k - y_k)f'(y_{ink}) \\ f'(y_{in}) &= f(y_{in})[1-f(y_{in})] \\ &= 0.5227(1-0.5227) \\ &= 0.2495\end{aligned}$$

$$\begin{aligned}\delta_1 &= (1-0.5227)(0.2495) \\ &= 0.1191\end{aligned}$$

calculates its weight correction term (used to update w_{jk} later),

$$\Delta w_{jk} = \alpha \delta_k z_j,$$

calculates its bias correction term (used to update w_{0k} later),

$$\Delta w_{0k} = \alpha \delta_k,$$

Calculate its weight correction for hidden to output layer

(Or Find the change in weights between hidden to output layer)

$$\Delta w_1 = \alpha \delta_1 z_1 = 0.25 \times 0.1191 \times 0.5498 = 0.0164$$

$$\Delta w_2 = \alpha \delta_1 z_2 = 0.25 \times 0.1191 \times 0.7109 = 0.02117$$

Bias correction $\Delta w_0 = \alpha \delta_1 = 0.25 \times 0.1191 = 0.02117$

Compute error portion δ_j between input to hidden layer ($j = 1$ to 2)

Step 7. Each hidden unit ($Z_j, j = 1, \dots, p$) sums its delta inputs (from units in the layer above),

$$\delta_{in_j} = \sum_{k=1}^m \delta_k w_{jk},$$

$$\delta_j = \sum_k \delta_k w_{jk} f'(z_{in_j})$$

$$\delta_{in_1} = \delta_1 w_1 = 0.1191 \times 0.4 = 0.04764$$

$$\delta_{in_2} = \delta_1 w_2 = 0.1191 \times 0.1 = 0.1191$$

$$\begin{aligned}f'(z_{in_1}) &= f(z_{in_1})[1-f(z_{in_1})] \\ f'(z_{in_1}) &= f(z_{in_1})[1-f(z_{in_1})] \\ &= 0.5498 \times (1-0.5498) \\ &= 0.2475\end{aligned}$$

$$\begin{aligned}f'(z_{in_2}) &= f(z_{in_2})[1-f(z_{in_2})] \\ &= 0.7109 \times (1-0.7109) \\ &= 0.2055\end{aligned}$$

$$f_1(x) = \frac{1}{1 + \exp(-x)},$$

$$f'_1(x) = f_1(x)[1 - f_1(x)].$$

multiplies by the derivative of its activation function to calculate its error information term,

$$\delta_j = \delta_{in_j} f'(z_{in_j}),$$

$$\delta_1 = \delta_{in_1} f'(z_{in_1}) = 0.04764 \times 0.2475 = 0.0118$$

$$\delta_2 = \delta_{in_2} f'(z_{in_2}) = 0.1191 \times 0.2055 = 0.00245$$

Find change in weights between input to hidden layer

calculates its weight correction term (used to update v_{ij} later),

$$\Delta v_{ij} = \alpha \delta_j x_i,$$

and calculates its bias correction term (used to update v_{0j} later),

$$\Delta v_{0j} = \alpha \delta_j.$$

$$\Delta v_{11} = \alpha \delta_1 x_1 = 0.25 \times 0.0118 \times 0 = 0$$

$$\Delta v_{21} = \alpha \delta_1 x_2 = 0.25 \times 0.0118 \times 1 = 0.00295$$

$$\Delta v_{01} = \alpha \delta_1 = 0.25 \times 0.0118 = 0.00295$$

$$\Delta v_{12} = \alpha \delta_2 x_1 = 0.25 \times 0.00245 \times 0 = 0$$

$$\Delta v_{22} = \alpha \delta_2 x_2 = 0.25 \times 0.00245 \times 1 = 0.0006125$$

$$\Delta v_{02} = \alpha \delta_2 = 0.20 \times 0.00245 = 0.0006125$$

Compute the final weight of the network

$$v_{11}(\text{new}) = v_{11}(\text{old}) + \Delta v_{11} = 0.6 + 0 = 0.6$$

$$v_{12}(\text{new}) = v_{12}(\text{old}) + \Delta v_{12} = -0.3 + 0 = -0.3$$

$$v_{21}(\text{new}) = v_{21}(\text{old}) + \Delta v_{21} = -0.1 + 0.00245 = -0.04705$$

$$v_{22}(\text{new}) = v_{22}(\text{old}) + \Delta v_{22} = 0.4 + 0.0006125 = 0.4006125$$

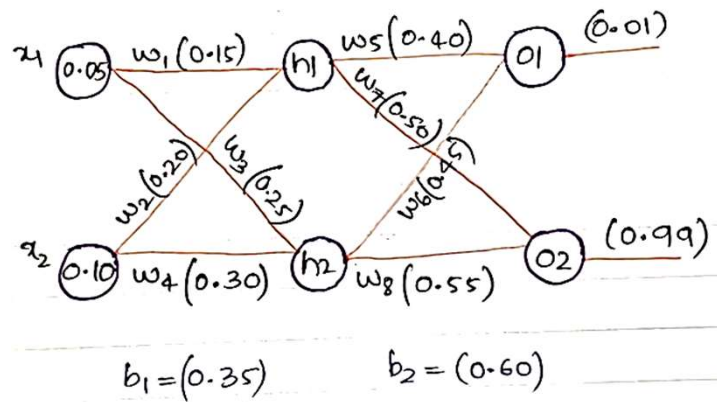
$$v_{01}(\text{new}) = v_{01}(\text{old}) + \Delta v_{01} = 0.3 + 0.00295 = 0.30295$$

$$v_{02}(\text{new}) = v_{02}(\text{old}) + \Delta v_{02} = 0.5 + 0.006125 = 0.5066125$$

$$w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 0.4 + 0.0164 = 0.4164$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = -0.1 + 0.02117 = 0.12117$$

$$w_0(\text{new}) = w_0(\text{old}) + \Delta w_0 = -0.2 + 0.02978 = -0.17022$$

Example # 2:*** BACK PROPAGATION ALGORITHM:**

(Backward propagation of error)

When error occurs, we go in backward direction
i.e. **output** $\rightarrow$ **hidden** $\rightarrow$ **input** layers.

Part 1: Calculate forward Propagation Error.

1) calculate h_1 (in and out)

$$h_1(\text{in}) = w_1 x_1 + w_2 x_2 + b_1$$

$$= 0.15 \times 0.05 + 0.2 \times 0.1 + 0.35$$

$$= 0.377$$

$$h_1(\text{out}) = \frac{1}{1 + e^{-h_1(\text{in})}}$$

$$= \frac{1}{1 + e^{-(0.377)}} = 0.5932$$

2) calculate h_2 (in and out)

$$h_2(\text{in}) = x_1 w_3 + x_2 w_4 + b_1$$

$$= 0.05 \times 0.25 + 0.10 \times 0.3 + 0.35 \quad h_2(\text{out}) = 0.596$$

$$= 0.0125 + 0.03 + 0.35 = \underline{\underline{0.3925}}$$

3) calculate o_1 (in and out)

$$o_1(\text{in}) = h_1(\text{out}) \times w_5 + h_2(\text{out}) \times w_6 + b_2$$

$$= 0.593 \times 0.4 + 0.596 \times 0.45 + 0.6$$

$$= 1.105$$

$$o_1(\text{out}) = 1 / (1 + e^{-o_1(\text{in})}) = 0.7513$$

3) calculate o_1 (in and out)

$$o_1(\text{in}) = h_1(\text{out}) \times w_5 + h_2(\text{out}) \times w_6 + b_2$$

$$= 0.593 \times 0.4 + 0.596 \times 0.45 + 0.6$$

$$= 1.105$$

$$o_1(\text{out}) = 1 / (1 + e^{-o_1(\text{in})}) = 0.7513$$

4) calculate o_2 (in and out)

$$o_2(\text{in}) = h_1(\text{out}) \times w_7 + h_2(\text{out}) \times w_8 + b_2$$

$$= 0.5932 \times 0.50 + 0.5968 \times 0.55 + 0.6$$

$$= 1.22484$$

$$O_2(\text{out}) = 1 / (1 + e^{-O_2(\text{in})})$$

$$= 0.7729$$

5) calculate E_{Total}

$$E_{\text{Total}} = \sum \frac{1}{2} (\text{target} - O(P))^2$$

$$= E_{O1} + E_{O2}$$

$$= \frac{1}{2} (0.01 - 0.7513)^2 + \frac{1}{2} (0.99 - 0.7729)^2$$

$$= \frac{1}{2} (-0.7413)^2 + \frac{1}{2} (0.2171)^2$$

$$= 0.274 + 0.0235 = 0.29837 \text{ (approx)}$$

Part: 2: Calculating backward propagation error
(output layer $\rightarrow$ hidden layer)

w_5, w_6, w_7 and w_8 .

First let us adjust w_5

$$w_5^* = w_5 - \eta \frac{\partial E_{\text{Total}}}{\partial w_5}$$

$$\frac{\partial E_{\text{Total}}}{\partial w_5} = \frac{\partial E_{\text{Total}}}{\partial \text{out}_{O1}} \times \frac{\partial \text{out}_{O1}}{\partial \text{net}_{O1}} \times \frac{\partial \text{net}_{O1}}{\partial w_5}$$

$$\frac{\partial E_{\text{Total}}}{\partial \text{out}_{O1}} = \text{out}_{O1} - \text{target}_{O1} = 0.751365 - 0.01$$

$$= 0.741365$$

Training Algorithm of Backpropagation

An **epoch** is one cycle through the entire set of training vectors. Typically, many epochs are required for training a backpropagation neural net.

The foregoing algorithm updates the weights after each training pattern. A common variation is **batch updating**, in which weight updates are accumulated over an entire epoch.

Choice of initial weights and biases

The choice of initial weights influence whether the net reaches a global (or only a local) minimum of the error, and, if so, how quickly it converges.

The update of the weight between two units depends on both the derivative of the upper unit's activation function and the activation function of the lower unit.

For this reason, it is important to avoid choices of initial weights that would make it likely that either activations or derivatives of activations are zero.

- The values for the **initial weights must not be too large** or the initial input signals to each hidden or output unit will be likely to fall in the region where the derivative of the sigmoid function has a very small value (so-called saturation region).

Choice of initial weights and biases (Contd.)

On the other hand, **if the initial weights are too small, the net input to a hidden or output unit will be close to zero, which also causes extremely slow learning.**

A common procedure is to initialize the weights (and biases) to **random values between -0.5 and 0.5 (or between -1 and 1 or some other suitable interval).**

The values may be positive or negative because the final weights after training may be of either sign also.

Dr. M.A.R. Khan, ICE, NSTU

Choice of initial weights and biases

Nguyen-Widrow Initialization:

The following simple modification of the **common random weight initialization just gives much faster learning.**

The approach is based on a geometrical analysis of the response of the hidden neurons to a single input; the analysis is extended to the case of several inputs by using Fourier transforms.

Weights from the hidden units to the output units (and biases on the output units) are initialized to random values between -0.5 and 0.5, as is commonly the case.

Nguyen-Widrow Initialization (Contd.)

The initialization of the weights from the input units to the hidden units is designed to improve the ability of the hidden units to learn.

This is accomplished by distributing the initial weights and biases so that, for each input pattern, it is likely that the net input to one of the hidden units will be in the range in which that hidden neuron will learn most readily.

The definitions are as follows:

n number of input units
 p number of hidden units
 β scale factor:

$$\beta = 0.7 (p)^{1/n} = 0.7 \sqrt[n]{p}$$

The procedure consists of the following simple steps:

for each hidden unit ($j = 1, \dots, p$):

Initialize its weight vector (from the input units):

Dr. M.A.R. Khan, ICE, NSTU

Nguyen-Widrow Initialization (Contd.)

$v_{ij}(\text{old}) = \text{random number between } -0.5 \text{ and } 0.5 \text{ (or between } -\gamma \text{ and } \gamma).$

Compute $\|\mathbf{v}_j(\text{old})\| = \sqrt{V_{1j}(\text{old})^2 + V_{2j}(\text{old})^2 + \dots + V_{nj}(\text{old})^2}$

Reinitialize weights:

$$v_{ij} = \frac{\beta v_{ij}(\text{old})}{\|\mathbf{v}_j(\text{old})\|}.$$

Set bias:

$v_{0j} = \text{random number between } -\beta \text{ and } \beta.$

The Nguyen-Widrow analysis is based on the activation function

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}},$$

which is closely related to the bipolar sigmoid activation function

How long to train the net

It is not necessary to continue training until the squared error actually reaches a minimum. Hecht-Nielsen (1990) suggests using **two sets of data during training**:

- a set of training patterns and
- a set of training-testing patterns.

These two sets are disjoint.

Weight adjustments are based on the training patterns; however, at intervals during training, the error is computed using the training-testing patterns.

As long as the error for the training-testing patterns decreases, training continues.

When the error begins to increase, the net is starting to memorize the training patterns too specifically. At this point, training is terminated.

Number of Hidden Layers (Contd.)

One hidden is sufficient for a backpropagation net to approximate any continuous mapping from the input patterns to the output patterns to an arbitrary degree of accuracy. However, two hidden layers may make training easier in some situations.

In general, adjusting the number of layers and the number of nodes of hidden layers of the network structure is carried out using a trial-and-error approach.

However, for most appliances one hidden layer is enough, while for more complicated applications the simulation usually takes place using two hidden layers. The existence of more hidden layers than are necessary complicates the network, resulting in a **low speed of convergence** during training and large error during operation.

How many training pairs there should be. A relationship among the number of training patterns available, P , the number of weights to be trained, W , and the accuracy of classification expected, e , is summarized in the following rule of thumb. For a more precise statement, with proofs, see Baum and Haussler, (1989). The question to be answered is “Under what circumstances can I be assured that a net which is trained to classify a given percentage of the training patterns correctly will also classify correctly testing patterns drawn from the same sample space?” Specifically, if the net is trained to classify the fraction $1 - (e/2)$ of the training patterns correctly, where $0 < e \leq 1/8$, can I be sure that it will classify $1 - e$ of the testing patterns correctly? The answer is that if there are enough training patterns, the net will be able to generalize as desired (classify unknown testing patterns correctly). Enough training patterns is determined by the condition

$$\frac{W}{P} = e,$$

or

$$P = \frac{W}{e}.$$

For example, with $e = 0.1$, a net with 80 weights will require 800 training patterns to be assured of classifying 90% of the testing patterns correctly, assuming that the net was trained to classify 95% of the training patterns correctly.

Problem # 2: Consider a 2-4-1 backprop net with two input units, four hidden units in one hidden layer and one output unit to solve the XOR problem. Use binary data representation for this problem, with the binary sigmoid for the activation function on all hidden and output units. Random values of initial weights are chosen between -0.5 and 0+.5

The initial weights to the hidden layer are :

-0.3378 0.2771 0.2859 -0.3329 (biases to the four hidden units)
 0.1970 0.3191 -0.1448 0.3594 (weights from the first input unit)
 0.3099 0.1904 -0.0347 -0.4861 (weights from the second input unit).

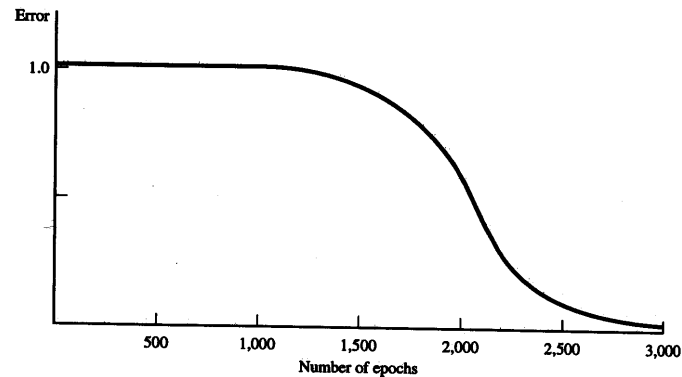
The initial weights from the hidden units to the output unit are:

-0.1401 (bias on the output unit)
 0.4919 (weight from the first hidden unit)
 -0.2913 (weight from the second hidden unit)
 -0.3979 (weight from the third hidden unit)
 0.3581 (weight from the fourth hidden unit)

Problem # 2 (Contd.)

The learning rate is 0.02.

The training continued until the total squared error for the four training patterns is less than 0.05.



Total squared error for binary representation of XOR problem.

In the above figure, for binary case, training took almost 3,000 epochs.

Problem # 3: Consider a 2-4-1 backprop net with two input units, four hidden units in one hidden layer and one output unit to solve the XOR problem. Use bipolar data representation bipolar sigmoid activation function on all hidden and output units. Random values of initial weights are chosen between -0.5 and 0+.5

The initial weights to the hidden layer are :

-0.3378	0.2771	0.2859	-0.3329	(biases to the four hidden units)
0.1970	0.3191	-0.1448	0.3594	(weights from the first input unit)
0.3099	0.1904	-0.0347	-0.4861	(weights from the second input unit).

Example 2 (Contd.)

The initial weights from the hidden units to the output unit are:

- 0.1401 (bias on the output unit)
- 0.4919 (weight from the first hidden unit)
- 0.2913 (weight from the second hidden unit)
- 0.3979 (weight from the third hidden unit)
- 0.3581 (weight from the fourth hidden unit)

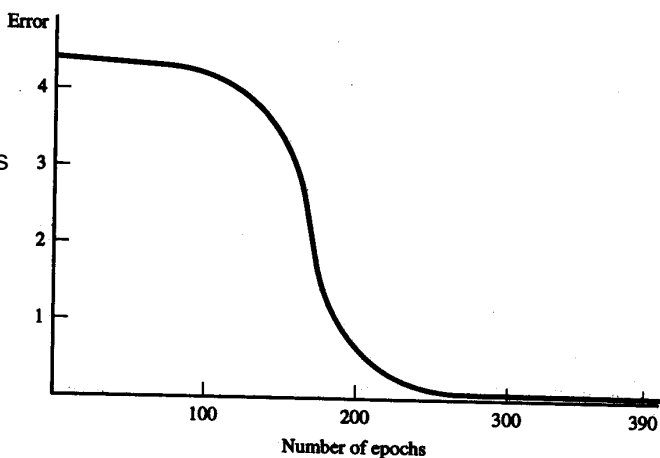
The learning rate is 0.02.

Training continued until the total squared error is less than 0.05

Example 2 (Contd.)

Using bipolar representation for the training data for the XOR problem and the bipolar sigmoid for the activation function for the hidden and output units gives much faster training than for binary case as illustrated in the above example # 1.

Here training takes only 387 epochs.



Total squared error for bipolar representation of XOR problem.

Momentum

In backpropagation with momentum, the weight change is in a direction that is a combination of the current gradient and the previous gradient.

This is a modification of gradient descent whose advantages arise chiefly when some training data are very different from the majority of the data.

It is desirable to use a small learning rate to avoid a major disruption of the direction of learning when a very unusual pair of training patterns is presented.

However, it is also preferable to maintain training at a fairly rapid pace as long as the training data are relative similar.

Dr. M.A.R. Khan, ICE, NSTU

Momentum

Convergence is sometimes faster if a momentum term is added to the weight update formulas.

In backpropagation with momentum, the new weights for training steps $t=1$ are based on the weights at training steps t and $t-1$.

The weight update formulas for backpropagation with momentum are

$$w_{jk}(t + 1) = w_{jk}(t) + \alpha \delta_k z_j + \mu [w_{jk}(t) - w_{jk}(t - 1)],$$

or

$$\Delta w_{jk}(t + 1) = \alpha \delta_k z_j + \mu \Delta w_{jk}(t),$$

and

$$v_{ij}(t + 1) = v_{ij}(t) + \alpha \delta_j x_i + \mu [v_{ij}(t) - v_{ij}(t - 1)],$$

or

$$\Delta v_{ij}(t + 1) = \alpha \delta_j x_i + \mu \Delta v_{ij}(t),$$

Momentum

where the momentum parameter μ is constrained to be in the range from 0 to 1.

Momentum allows the net to make reasonably large weight adjustment as long as the corrections are in the same general direction for several patterns, while using a smaller learning rate to prevent a large response to the error from any one training pattern. It also reduces the likelihood that the net will find weights that are a local, but not global, minimum. When using momentum, the net is proceeding not in the direction of the gradient, but in the direction of a combination of the current gradient and the previous direction of weight correction.

Example

A backpropagation net with momentum for the XOR function: bipolar representation.

Using the same initial weights and architecture as in examples 1 to 4 and target values of +1.0 and -1.0, adding momentum (0.9) with a learning rate as before (0.2) reduces the training requirements from 387 epochs to 38 epochs.

Batch updating of weights

In some cases it is advantageous to accumulate the weight correction terms for several patterns (or even an entire epoch if there are not too many patterns) and make a single weight adjustment (equal to the average of the weight correction terms) for each weight rather than updating the weights after each pattern is presented. This procedure has a smoothing effect on the correction terms. In some cases, this smoothing may increase the chances of convergence to a local minimum.

Thank you